CSE755: Practice Problems and Solutions (Lecture 1 - 3)

Contents

How to read these solutions 2

Problem 1 (Week 1): What a classifier can and cannot do 3

Problem 2 (Week 1): Maximum likelihood for a Bernoulli model 4

Problem 3 (Week 1): Why maximum likelihood equals minimizing KL 5

Problem 4 (Week 1): Latent variables and the hard integral 7

Problem 5 (Week 1): A two-component Gaussian mixture 8

Problem 6 (Week 2): Chain rule on a 2x2 image in two orders 11

Problem 7 (Week 2): Likelihood and bits per dimension 12

Problem 8 (Week 2): Teacher forcing, ancestral sampling, exposure bias 13

Problem 9 (Week 2): PixelCNN masks and the blind spot 14

Problem 10 (Week 2): Colour channels inside one pixel 16

Week 3 setup: the pen story 18

Problem 11 (Week 3): Exact marginal and posterior 18

Problem 12 (Week 3): Deriving the ELBO and its gap 19

Problem 13 (Week 3): The Gaussian KL and its derivatives 21

Problem 14 (Week 3): Reparameterization and reconstruction gradients 23

Problem 15 (Week 3): Two encoders, two ELBOs 24

How to read these solutions

Every problem has four parts.

1. **Question**: the problem as written on the practice sheet.
2. **What the question is asking**: the same question in plain words.
3. **Solution and explanation**: how to think about it, then the answer, step by step.
4. **Concepts used**: the names of the lecture ideas the solution relied on.

A few symbols appear everywhere, so here they are once:

- x is the thing a generative model produces or scores: an image, a sentence, a sound clip.
- y is a label, such as "cat" or "dog".
- z is a *latent* (hidden) variable: something the model uses internally but that is never in the dataset, such as the hidden "style" of an image.
- θ (theta) is the list of numbers the model learns (its parameters, i.e. the network weights).
- $p(a)$ is "the probability of a". $p(a \mid b)$ is "the probability of a if we already know b".
- log means the natural logarithm unless it is written $\log_2$.

The practice sheet gives two constants used in Week 3:

$$\tfrac{1}{2}\log(2\pi) \approx 0.919, \qquad \log 2 \approx 0.693.$$

Problem 1 (Week 1): What a classifier can and cannot do

Question

A trained model provides only $p(y \mid x)$, and nothing else.

1. Name one query this model can answer, and one query it cannot answer.
2. From the product rule $p(x, y) = p(y \mid x)\, p(x) = p(x \mid y)\, p(y)$, which extra distribution do you need in order to sample a new image of class $y = \text{cat}$?

What the question is asking

$p(y \mid x)$ reads "the probability of label y, given image x". A model that only gives you this is a *classifier*: you show it an image and it tells you how likely each label is.

Part 1 asks: what kind of question can you put to this classifier, and what kind can you not?

Part 2 asks: if you want to *create* a brand-new cat picture, what extra piece of information is missing? The product rule is the hint.

Solution and explanation

Part 1

The idea. The classifier always needs an image as its input. It only ever answers "given this image, which label?". It has never learned what images themselves look like.

A query it can answer. "Here is a photo. What is the probability that it shows a cat?" That is

$$p(y = \text{cat} \mid x).$$

A query it cannot answer. "How likely is this photo to be a real-looking image at all?" That is $p(x)$, the probability of the image itself. The classifier has no number for this. For the same reason it cannot draw a new image either.

Why it cannot. Picture a cat-vs-dog classifier given a picture of pure TV static. It must still output something like "70% cat, 30% dog", because its outputs always add up to 1 over the labels. It has no way to say "this image is nonsense". Knowing how to tell cats from dogs is not the same as knowing what a realistic image looks like.

Part 2

The idea. To draw a cat image we need $p(x \mid y = \text{cat})$: "which images are likely, given that the label is cat". Then we draw an image from that distribution. The classifier gives us the reverse conditional, $p(y \mid x)$. The product rule tells us how to turn one into the other.

Step 1. Start from the product rule. It writes the same joint probability in two ways:

$$p(y \mid x)\, p(x) = p(x \mid y)\, p(y).$$

Step 2. Divide both sides by $p(y)$ to get the quantity we want:

$$p(x \mid y) = \frac{p(y \mid x)\, p(x)}{p(y)}.$$

Step 3. Look at the right-hand side and check what we have.

- $p(y \mid x)$: we have it (it is the classifier).

- $p(y)$: how common cats are overall. If we had $p(x)$, we could get this by adding up $p(y \mid x)\, p(x)$ over all images.
- $p(x)$: the probability of the image itself. **This is the missing piece.**

Answer. You need $p(x)$, the distribution over images (or, equivalently, a model of $p(x \mid y)$ directly). In practice nobody computes Step 2 over all possible images. You simply train a class-conditional generative model $p(x \mid y)$, which generates images of a requested class.

Plain-English summary. The classifier knows how to *judge*, not how to *draw*. To draw you need a model of what images look like.

Concepts used

Joint, marginal and conditional distributions; product rule; Bayes' rule; discriminative vs. generative models (Lecture 1).

Problem 2 (Week 1): Maximum likelihood for a Bernoulli model

Question

i.i.d. Bernoulli observations with $p(x = 1 \mid \theta) = \theta$. The dataset has $N_1 = 8$ ones and $N_0 = 2$ zeros.

1. Write the log-likelihood $\ell(\theta)$.
2. Compute $\partial \ell / \partial \theta$ and solve $\partial \ell / \partial \theta = 0$ for $\hat{\theta}_{\mathrm{ML}}$.
3. In one sentence: in the likelihood $L(\theta) = p(\mathcal{D} \mid \theta)$, what is held fixed, and what varies?

What the question is asking

A *Bernoulli* variable can only be 0 or 1. Here is a generative example. A tiny model generates black-and-white images of handwritten digits, and we look at the centre pixel only. The pixel is 1 (ink) or 0 (blank). The model has one knob, θ: the probability that it draws ink there.

We looked at 10 training images. The centre pixel was ink in 8 of them and blank in 2. *i.i.d.* (independent and identically distributed) means each image was produced separately, by the same model.

The question asks us to find the value of θ that makes this observed data most probable. That value is the *maximum likelihood estimate*.

Solution and explanation

Part 1: the log-likelihood

The idea. The likelihood is the probability the model gives to the whole dataset. Because the images are independent, this is a product of one probability per image:

- each ink pixel contributes θ;
- each blank pixel contributes $1 - \theta$.

So the likelihood is

$$L(\theta) = \theta^8 \, (1 - \theta)^2.$$

Multiplying many small numbers gives tiny numbers that computers round to zero. So we take the logarithm, which turns the product into a sum. It does not change where the maximum is, because the

logarithm always increases when its input increases.

$$\ell(\theta) = \log L(\theta) = N_1 \log \theta + N_0 \log(1-\theta) = 8 \log \theta + 2 \log(1-\theta).$$

Part 2: the derivative and the best θ

The idea. At the top of a hill the slope is zero. So we take the slope (derivative) of ℓ with respect to θ and find where it equals zero.

Using $\frac{d}{d\theta} \log \theta = \frac{1}{\theta}$ and $\frac{d}{d\theta} \log(1-\theta) = -\frac{1}{1-\theta}$:

$$\frac{\partial \ell}{\partial \theta} = \frac{8}{\theta} - \frac{2}{1-\theta}.$$

Set it to zero and solve:

$$\frac{8}{\theta} = \frac{2}{1-\theta} \implies 8(1-\theta) = 2\theta \implies 8 = 10\theta \implies \hat{\theta}_{\mathrm{ML}} = 0.8.$$

Is it really a maximum, not a minimum? The second derivative is

$$-\frac{8}{\theta^2} - \frac{2}{(1-\theta)^2},$$

which is negative for every θ between 0 and 1. So the curve bends downward, and the point is a peak.

What it means. The answer is simply $N_1/N = 8/10$, the fraction of training images with ink there. Maximum likelihood says: "copy the frequency you saw in the data". Deep generative models use exactly the same principle. The only difference is that the formula is too complicated to solve by hand, so they climb the hill with gradient steps.

Part 3: what is fixed and what varies

Answer. In the likelihood, the data $\mathcal{D}$ is held fixed (it has already been observed) and the parameter θ varies; we ask "which setting of the model makes this fixed data most probable?".

Compare this with an ordinary probability $p(x \mid \theta)$. There the model setting θ is fixed, and we ask how probable different outputs x are. Same formula, read in the opposite direction. Because of this, the likelihood is *not* a probability distribution over θ: it does not have to add up to 1 across all values of θ.

Concepts used

Bernoulli distribution; i.i.d. assumption; likelihood vs. probability; log-likelihood; maximum likelihood estimation (MLE) (Lecture 1, Week 1 hand notes: "Worked example: Bernoulli MLE").

Problem 3 (Week 1): Why maximum likelihood equals minimizing KL

Question

Show that maximizing $\mathbb{E}_{p_{\mathrm{data}}}[\log p_\theta(x)]$ is equivalent to minimizing $\mathrm{KL}(p_{\mathrm{data}} \,\|\, p_\theta)$. Start from $\mathrm{KL}(p\|q) = \mathbb{E}_p[\log p - \log q]$ and name the term that does not depend on θ.

What the question is asking

There are two distributions here.

- p_{data}: the real, unknown process that produced the training data. For example, all the cat photos that exist in the world.
- p_θ: our model, which we can adjust by changing θ.

$\mathbb{E}_{p_{\text{data}}}[\log p_\theta(x)]$ means: take real images, ask the model how likely it finds each one (in log form), and average. That is "average log-likelihood on real data", which is what training maximizes.

$\text{KL}(p_{\text{data}} \| p_\theta)$ (Kullback–Leibler divergence) is a number measuring how different the model is from reality. It is 0 when they are identical and positive otherwise.

The question asks us to show, with algebra, that making the first quantity bigger is *the same thing* as making the second smaller.

Solution and explanation

The idea. Write out the KL. It splits into two pieces. One piece is exactly the training objective (with a minus sign). The other piece depends only on the real data, so the model cannot change it.

Step 1. Write the definition with $p = p_{\text{data}}$ and $q = p_\theta$:

$$\text{KL}(p_{\text{data}} \| p_\theta) = \mathbb{E}_{p_{\text{data}}}\big[\log p_{\text{data}}(x) - \log p_\theta(x)\big].$$

Step 2. An average of a difference is the difference of the averages:

$$\text{KL}(p_{\text{data}} \| p_\theta) = \underbrace{\mathbb{E}_{p_{\text{data}}}[\log p_{\text{data}}(x)]}_{\text{no } \theta \text{ here}} - \underbrace{\mathbb{E}_{p_{\text{data}}}[\log p_\theta(x)]}_{\text{training objective}}.$$

Step 3. Name the first term. The *entropy* of the data is

$$H(p_{\text{data}}) = -\mathbb{E}_{p_{\text{data}}}[\log p_{\text{data}}(x)],$$

so the first term is $-H(p_{\text{data}})$. Rearranging gives the formula from the lecture:

$$\mathbb{E}_{p_{\text{data}}}[\log p_\theta(x)] = -H(p_{\text{data}}) - \text{KL}(p_{\text{data}} \| p_\theta).$$

Step 4. Conclude. $H(p_{\text{data}})$ measures how varied the real images are. It is a property of the world, not of the model, so it is a constant as far as θ is concerned. That leaves only $-\text{KL}$ to change. Making the left side bigger can only happen by making the KL smaller. So the two goals are the same.

The term that does not depend on θ: the entropy of the data, $H(p_{\text{data}})$ (equivalently, $\mathbb{E}_{p_{\text{data}}}[\log p_{\text{data}}]$).

Intuition. Think of a text generator trained on news articles. Every time it gives a real article a higher probability, it has moved its idea of "what text looks like" closer to what real news looks like. That is exactly what "smaller KL" means.

Why the direction of KL matters (from the lecture). $\text{KL}(p_{\text{data}} \| p_\theta)$ punishes the model heavily if it gives almost zero probability to something that really occurs. So a maximum-likelihood image model tries to cover every kind of real image, even rare ones. This is called *mode covering*.

Link to practice. We cannot compute an average over *all* real images. We use the training set instead: $\frac{1}{N}\ell(\theta) = \frac{1}{N}\sum_n \log p_\theta(x^{(n)})$ is an estimate of $\mathbb{E}_{p_{\text{data}}}[\log p_\theta(x)]$.

Concepts used

KL divergence; entropy; expectation; maximum likelihood as KL minimization; mode covering (Lecture 1, slide "Maximum likelihood estimation").

Problem 4 (Week 1): Latent variables and the hard integral

Question

A continuous latent and a neural decoder define $p_\theta(x) = \int p_\theta(x \mid z)\, p_\theta(z)\, dz$.

1. Write the joint $p_\theta(x, z)$ and the true posterior $p_\theta(z \mid x)$.
2. Why is the integral typically intractable when the decoder is a multilayer network?
3. A classmate claims: if $p(x_1, x_2 \mid z) = p(x_1 \mid z)\, p(x_2 \mid z)$, then $p(x_1, x_2) = p(x_1)p(x_2)$. Is this true?

What the question is asking

This is a *latent-variable model*. It generates in two steps:

1. pick a hidden code z from a prior $p_\theta(z)$ (think: the hidden "style" of a face: pose, lighting, hair);
2. a neural network called the *decoder* turns z into an image, described by $p_\theta(x \mid z)$.

The probability of an image, $p_\theta(x)$, is found by adding up (integrating) over every possible hidden code that could have produced it.

Part 1 asks for two formulas: the probability of (image, code) together, and the probability of the code once we have seen the image. Part 2 asks why the integral is so hard. Part 3 asks whether "independent given z" implies "independent overall".

Solution and explanation

Part 1: joint and posterior

Joint. "Pick z, then make x from z" is a product of the two steps:

$$p_\theta(x, z) = p_\theta(x \mid z)\, p_\theta(z).$$

Posterior. The posterior answers the reverse question: "I have this image. Which hidden codes probably produced it?". Bayes' rule gives

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)} = \frac{p_\theta(x \mid z)\, p_\theta(z)}{\int p_\theta(x \mid z')\, p_\theta(z')\, dz'}.$$

Here z' is just a dummy name for "every possible code" inside the integral. Notice that the bottom of the fraction is the same hard integral as $p_\theta(x)$. So the posterior is hard whenever $p_\theta(x)$ is hard.

Part 2: why the integral is intractable

The idea. An integral can be done by hand only when the formula inside has a known "undo" (an antiderivative). Simple shapes, like a Gaussian inside a Gaussian, have one. A neural network has none.

- The decoder is many layers of "multiply, add, apply a nonlinearity" stacked together. When z is pushed through all of that, the resulting formula has no closed-form integral. The lecture's phrase: the integral "does not factor through a nonlinear net".
- We also cannot simply try every z on a computer. Suppose z has 100 numbers and we try only 10 values for each. That is 10^{100} combinations, far too many to compute.

So $p_\theta(x)$, and therefore the posterior, cannot be computed exactly. This single fact shapes the rest of the course: Week 2 avoids latents altogether; Week 3 (VAE) keeps latents and uses a lower bound instead.

Part 3: the classmate's claim

Answer: false. Independence *given* z does not give independence *without* z.

Intuition with a text generator. Let z be the hidden *topic* of a generated sentence: sports or cooking. Let x_1 and x_2 be the first and second words.

- Once you know the topic is sports, the first word gives no extra hint about the second; both are just "some sports word". That is conditional independence.
- If you do *not* know the topic and the first word is "goal", you can guess the topic is sports, so the second word is probably sporty too. The first word told you something about the second. They are dependent.

The shared hidden cause ties the words together.

The maths. Removing z means summing over it:

$$p(x_1, x_2) = \sum_z p(x_1 \mid z)\, p(x_2 \mid z)\, p(z).$$

A sum of products does not split into (something with only x_1) times (something with only x_2), so in general $p(x_1, x_2) \neq p(x_1)p(x_2)$.

A small numerical check. Topic z is sports or cooking with probability 0.5 each. Each word is a "sports word" with probability 0.9 under sports and 0.1 under cooking.

$$p(\text{both sports words}) = 0.5(0.9)(0.9) + 0.5(0.1)(0.1) = 0.405 + 0.005 = 0.41.$$

$$p(\text{word 1 sporty}) = 0.5(0.9) + 0.5(0.1) = 0.5, \qquad p(x_1)p(x_2) = 0.5 \times 0.5 = 0.25.$$

Since $0.41 \neq 0.25$, the words are not independent.

This is the whole reason latent variables are useful: a small hidden cause creates the correlations among the many pixels or words of the output.

Concepts used

Latent variable; generative story; joint, marginal likelihood (evidence) and posterior; Bayes' rule; intractability of the marginal; conditional independence vs. independence (Lecture 1, slides "Marginal likelihood, posterior, and the hard integral" and "Conditional independence given the latent").

Problem 5 (Week 1): A two-component Gaussian mixture

Question

A two-component mixture has $\pi_1 = 0.3$, $\pi_2 = 0.7$, $\mu_1 = 0$, $\mu_2 = 2$, and $\sigma^2 = 1$ for both components, where

$$\mathcal{N}(x; \mu, 1) = \frac{1}{\sqrt{2\pi}}\, e^{-(x-\mu)^2/2}.$$

1. Write $p(x)$ and the responsibility $r_1(x) = p(z = 1 \mid x)$.
2. Evaluate $r_1(0)$. Leave the two Gaussian densities visible before you divide. Use $\mathcal{N}(0; 0, 1) \approx 0.399$ and $\mathcal{N}(0; 2, 1) \approx 0.054$.

What the question is asking

A *mixture* model generates in two steps. Here is an audio example. A model generates the loudness x of short sound clips.

- First it secretly picks a type z: type 1 = quiet speech (chosen 30% of the time, $\pi_1 = 0.3$), type 2 = loud music (chosen 70% of the time, $\pi_2 = 0.7$).
- Then it draws the loudness from a bell curve for that type: centred at 0 for speech ($\mu_1 = 0$), centred at 2 for music ($\mu_2 = 2$), both with spread (variance) 1.

$\mathcal{N}(x; \mu, 1)$ is the height of the bell curve with centre μ at the point x.

Part 1 asks for the overall probability of a loudness value, and for the *responsibility*: given a clip with loudness x, how likely is it to be speech? Part 2 asks for that number at $x = 0$.

Solution and explanation

Part 1: $p(x)$ and $r_1(x)$

$p(x)$. A clip with loudness x can come from either type. Add up both routes, each weighted by how often that type is chosen:

$$p(x) = \pi_1 \mathcal{N}(x; 0, 1) + \pi_2 \mathcal{N}(x; 2, 1) = 0.3\,\mathcal{N}(x; 0, 1) + 0.7\,\mathcal{N}(x; 2, 1).$$

This is the "sum over z" from Problem 4, and here it is easy because there are only two values of z.

$r_1(x)$. Use Bayes' rule: (prior for type 1) $\times$ (how well type 1 explains x), divided by the total over both types:

$$r_1(x) = p(z = 1 \mid x) = \frac{\pi_1 \mathcal{N}(x; 0, 1)}{\pi_1 \mathcal{N}(x; 0, 1) + \pi_2 \mathcal{N}(x; 2, 1)} = \frac{0.3\,\mathcal{N}(x; 0, 1)}{0.3\,\mathcal{N}(x; 0, 1) + 0.7\,\mathcal{N}(x; 2, 1)}.$$

Notice the bottom is exactly $p(x)$. The responsibility is a *soft* assignment. Instead of saying "this clip is speech", it says "this clip is speech with probability r_1".

Part 2: $r_1(0)$

The idea. At $x = 0$ the clip sits exactly at the centre of the speech curve and two units away from the music curve. Speech explains it much better, but music is chosen more often. The responsibility weighs these two forces against each other.

Step 1. Plug in the given curve heights, keeping them visible:

$$r_1(0) = \frac{0.3 \times \mathcal{N}(0; 0, 1)}{0.3 \times \mathcal{N}(0; 0, 1) + 0.7 \times \mathcal{N}(0; 2, 1)} = \frac{0.3 \times 0.399}{0.3 \times 0.399 + 0.7 \times 0.054}.$$

Step 2. Multiply:

$$0.3 \times 0.399 = 0.1197, \qquad 0.7 \times 0.054 = 0.0378.$$

Step 3. Divide:

$$r_1(0) = \frac{0.1197}{0.1197 + 0.0378} = \frac{0.1197}{0.1575} \approx 0.76.$$

So $r_2(0) = 1 - 0.76 = 0.24$.

Reading the answer. Speech was the less common type (30%), but at $x = 0$ its curve is about 7 times taller than the music curve (0.399 vs. 0.054). The fit wins over the prior, and the model is about 76% sure the clip is speech.

Common mistake. Do not swap the log and the sum. $\log p(x) = \log\left(0.3\,\mathcal{N}_1 + 0.7\,\mathcal{N}_2\right)$, which is *not* $\log(0.3\,\mathcal{N}_1) + \log(0.7\,\mathcal{N}_2)$. This "log of a sum" is why mixtures are trained with EM rather than by setting a derivative to zero.

Concepts used

Gaussian mixture model; discrete latent variable; prior (mixing weights); marginal as a sum over z; Bayes' rule; posterior responsibility; preview of EM (Lecture 1, slides "Running example: a Gaussian mixture" and "Why EM appears").

Problem 6 (Week 2): Chain rule on a 2x2 image in two orders

Question

A 2×2 binary image is ordered in raster scan: $x_{11}, x_{12}, x_{21}, x_{22}$.

1. Write $p(x_{11}, x_{12}, x_{21}, x_{22})$ as a product of conditionals. Which variables may appear in the conditional for x_{21}?
2. Write the product that generates the bottom row first: $x_{21}, x_{22}, x_{11}, x_{12}$.
3. Are the two joints equal as functions? Are the factors equal?

What the question is asking

The image has 4 pixels. x_{rc} is the pixel in row r, column c:

$$\begin{pmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{pmatrix}$$

Raster order means reading like a book: top row left to right, then the next row.

The question asks us to write the probability of the whole image as "probability of pixel 1, times probability of pixel 2 given pixel 1, ...". Then it asks us to do it again in a different order, and to compare the two.

Solution and explanation

Part 1: raster order

The idea. The chain rule says: the probability of everything equals the probability of the first item, times the second given the first, times the third given the first two, and so on. This is exact, not an approximation. An autoregressive model draws an image this way, one pixel at a time, each pixel looking only at the ones already drawn.

$$p(x_{11}, x_{12}, x_{21}, x_{22}) = p(x_{11})\; p(x_{12} \mid x_{11})\; p(x_{21} \mid x_{11}, x_{12})\; p(x_{22} \mid x_{11}, x_{12}, x_{21}).$$

Which variables may appear in the conditional for x_{21}? Only x_{11} and x_{12}, the two pixels that come before it in raster order (the whole top row). x_{21} itself and x_{22} must not appear. If they did, the model would be peeking at the answer, and the product would no longer be a valid probability of the image.

Part 2: bottom row first

Same rule, new order $x_{21}, x_{22}, x_{11}, x_{12}$:

$$p(x_{11}, x_{12}, x_{21}, x_{22}) = p(x_{21})\; p(x_{22} \mid x_{21})\; p(x_{11} \mid x_{21}, x_{22})\; p(x_{12} \mid x_{21}, x_{22}, x_{11}).$$

Part 3: equal joints? equal factors?

The joints: yes, equal. The chain rule is an identity that holds for *every* order. Both products equal the same true $p(x_{11}, x_{12}, x_{21}, x_{22})$, for every one of the 16 possible images.

The factors: no, different. For example, the first factor is $p(x_{11})$ in one order and $p(x_{21})$ in the other. The top-left pixel is predicted with no context in the first order, but with the whole bottom row as context in the second. These are different questions with different answers.

Why this matters for a neural network. A neural autoregressive model must learn the factors. A different order means a different set of functions to learn. Some orders are easier than others: text is

naturally left-to-right, audio naturally follows time. A learned model is only an approximation, so two networks trained with different orders can end up giving slightly different joints in practice. The exact equality holds for the true conditionals. That is what the lecture means by "the order is part of the model".

Concepts used

Chain rule of probability; autoregressive factorization; raster order; context $x_{<i}$; causal constraint; "the order is part of the model" (Lecture 2, slides "Chain rule of probability" and "The order is part of the model").

Problem 7 (Week 2): Likelihood and bits per dimension

Question

An autoregressive model assigns probabilities 0.9, 0.8, 0.7, and 0.6 to the four true bits of a binary image.

1. Compute $p(x) = \prod_{i=1}^{4} p(x_i \mid x_{<i})$.
2. Compute $-\log_2 p(x)$ and bpd $= -\frac{1}{4}\log_2 p(x)$. Use $-\log_2 0.9 \approx 0.152$, $-\log_2 0.8 \approx 0.322$, $-\log_2 0.7 \approx 0.515$, and $-\log_2 0.6 \approx 0.737$.
3. A uniform binary model costs 1 bit per coordinate. Did this model beat that baseline? Say which direction is better.

What the question is asking

We have a real black-and-white image with 4 pixels. We feed it to the model in raster order. At each pixel, the model says how likely the true value was, given the earlier pixels. It said 0.9, 0.8, 0.7 and 0.6.

The question asks for three things: the probability of the whole image; how many bits a compressor based on this model would need, both in total and per pixel (*bits per dimension*, bpd); and whether that is better than a model that just guesses 50/50 for every pixel.

Solution and explanation

Part 1: probability of the image

The idea. By the chain rule, the probability of the whole image is the product of the per-pixel probabilities.

$$p(x) = 0.9 \times 0.8 \times 0.7 \times 0.6.$$

Step by step:

$$0.9 \times 0.8 = 0.72, \qquad 0.72 \times 0.7 = 0.504, \qquad 0.504 \times 0.6 = 0.3024.$$

So $p(x) \approx 0.302$.

Part 2: bits and bits per dimension

The idea. Information theory says an outcome with probability p needs about $-\log_2 p$ bits to store. A likely outcome is cheap to store and a surprising one is expensive. Because the logarithm turns a product into a sum, we can add the bits pixel by pixel:

$$-\log_2 p(x) = 0.152 + 0.322 + 0.515 + 0.737 = 1.726 \text{ bits}.$$

Bits per dimension divides by the number of pixels, $D = 4$, so models of different image sizes can be compared:

$$\text{bpd} = \frac{1.726}{4} \approx 0.43 \text{ bits per pixel.}$$

(Check: $\log_2 0.3024 \approx -1.726$, which matches.)

Part 3: did it beat the baseline?

The baseline. A "uniform" model knows nothing and gives every pixel probability 0.5. Each pixel costs $-\log_2 0.5 = 1$ bit, so the image costs 4 bits and the bpd is 1.

Answer. Yes. $0.43 < 1$, and **lower bpd is better**. Lower bpd means the model found the real image less surprising, so it gave the image higher probability and could compress it more. This model needs about 1.7 bits instead of 4.

For 8-bit colour images, each channel has 256 possible values, so the uniform baseline is $\log_2 256 = 8$ bpd. A useful image model must be below 8. For example, PixelCNN++ reaches about 2.92 bpd on CIFAR-10.

Concepts used

Chain rule; exact likelihood of an autoregressive model; negative log-likelihood in bits; bits per dimension (bpd); uniform baseline (Lecture 2, slides "Exact likelihood", "Bits per dimension" and "A 2×2 grayscale image, by hand").

Problem 8 (Week 2): Teacher forcing, ancestral sampling, exposure bias

Question

Answer in a few sentences each.

1. Define teacher forcing. Why is it valid maximum likelihood, and why does it not produce a sample?
2. Write the ancestral sampling recursion for $x_1, \ldots, x_D$.
3. Define exposure bias. Why do long sequences suffer more?

What the question is asking

An autoregressive model is used in two different ways.

- *Training*: we have real data and want the model to give it high probability.
- *Generation*: we have nothing and want the model to write something new.

The question asks how each one works, and what goes wrong because the two are different.

Solution and explanation

Part 1: teacher forcing

Definition. During training, when the model predicts position i, the earlier positions $x_{<i}$ it sees are the *true* values from the training example, never the model's own guesses. Example: training a text generator on the sentence "the cat sat on the mat". When predicting the 4th word, the model is shown the real first three words, "the cat sat", even if it would have guessed differently.

Why it is valid maximum likelihood. The chain rule says the log-probability of the real sentence is exactly

$$\log p_\theta(x) = \sum_{i=1}^{D} \log p_\theta(x_i \mid x_{<i}),$$

where every $x_{<i}$ is the real prefix. Teacher forcing computes exactly these terms, so the training loss is the true negative log-likelihood, with no approximation. A bonus: all the true prefixes are known in advance, so all D predictions can be computed in one parallel pass. That is why PixelCNN trains fast.

Why it does not produce a sample. Teacher forcing only *scores* a sequence that already exists. The context always comes from the data, and nothing is ever drawn by the model and fed back in. To create something new, the model must use its own outputs as context. That is a different procedure (Part 2).

Part 2: ancestral sampling

Idea. Draw the first item, then draw each next item from the model's prediction given everything drawn so far:

$$x_1 \sim p_\theta(x_1), \qquad x_i \sim p_\theta(x_i \mid x_1, \ldots, x_{i-1}) \quad \text{for } i = 2, 3, \ldots, D.$$

Here "$\sim$" means "is randomly drawn from". Each x_i is appended to the context before the next step. This is how a GPT-style model writes text token by token, and how a PixelCNN paints an image pixel by pixel. It needs D steps one after another, which is why sampling is slow (a $32 \times 32 \times 3$ image means 3072 steps).

Part 3: exposure bias

Definition. In training the model only ever sees perfect, real prefixes. In generation it sees its own prefixes, which may contain mistakes. So at generation time the model is fed contexts unlike anything it was trained on. This mismatch between training prefixes and sampling prefixes is *exposure bias*.

Why long sequences suffer more. Errors pile up. One odd word early on changes the context for every later word. The model may then make a second mistake, which changes the context again, and so on. A longer sequence has more steps where a first mistake can happen, and more later steps for that mistake to spread into.

A rough illustration (assuming, for simplicity, a 1% chance of an off-track token at each step, independently):

$$0.99^{10} \approx 0.90, \qquad 0.99^{1000} \approx 0.00004.$$

A 10-token reply stays fully on track about 90% of the time. A 1000-token essay almost never does. The same holds for a 256×256 image generated pixel by pixel. The lecture mentions *scheduled sampling* (sometimes feeding the model's own tokens during training) as one remedy, at the cost of no longer being pure maximum likelihood.

Concepts used

Teacher forcing; exact NLL via the chain rule; parallel training; ancestral sampling; sequential generation cost; exposure bias; scheduled sampling (Lecture 2, slides "Teacher forcing", "Ancestral sampling" and "Exposure bias").

Problem 9 (Week 2): PixelCNN masks and the blind spot

Question

A PixelCNN uses masked 3×3 convolutions in raster order.

1. Write the 3×3 zero/one patterns for Mask A and Mask B.
2. Which layer uses Mask A, and why may later layers use Mask B?
3. Define the blind spot. Is a blind-spot pixel a future pixel, or legal past the stack fails to see?

What the question is asking

A *convolution* is a small 3×3 window that slides over the image. At each pixel it combines that pixel and its 8 neighbours using 9 learned weights. PixelCNN must predict each pixel using only the pixels before it in raster order. So some of the 9 weights are forced to be zero. That zero/one pattern is called a *mask*: 1 means "allowed to look", 0 means "blocked".

The question asks for the two masks, where each is used, and what the "blind spot" is.

Solution and explanation

Part 1: the two masks

The idea. In the 3×3 window centred on the pixel being predicted:

- the whole row above is the past, so it is allowed;
- on the same row, the pixel to the left is the past (allowed) and the pixel to the right is the future (blocked);
- the whole row below is the future, so it is blocked;
- the centre is the only position that differs between the two masks.

$$\text{Mask A} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \qquad \text{Mask B} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Mask A blocks the centre. Mask B allows it.

Part 2: where each mask is used

Mask A: the first layer only. The first layer reads the actual image. At the centre position sits the very pixel value we are trying to predict. If the first layer could see it, the network would learn to copy its input. It would get near-perfect training scores, but the probabilities would be meaningless. It is like handing a student the answer sheet during the exam.

Mask B: every later layer. After the first layer, the value at the centre position is no longer the raw pixel. It is a *feature* that the first layer computed using only past pixels, because Mask A guaranteed that. Reading this feature cannot leak the answer, because it never contained the answer. Allowing the centre lets later layers reuse the information already collected at that position. With colour images, it also lets the green channel use features from red (red comes first in the R, G, B order).

Part 3: the blind spot

Definition. The blind spot is the set of pixels that are *legal context*, because they come earlier in raster order, but that a stack of masked 3×3 convolutions can never reach, no matter how deep the stack is.

Why it happens. Each layer may look one step up-right, but it may never look right on its own row. Follow the reach upward:

- in the row directly above the target, the stack can see at most 1 column to the right of the target;
- two rows above, at most 2 columns to the right;

- and so on.

Everything further to the right in those upper rows (the upper-right area) is in the past and allowed, but no chain of windows ever reaches it.

Answer to the last question. A blind-spot pixel is **legal past that the stack fails to see**. It is not a future pixel. Future pixels are blocked on purpose, which is correct. Blind-spot pixels are lost by accident, which is a weakness.

Generative example. Generating a picture of a horizontal power line: when painting a pixel on the line, the network would like to know where the line was, a few pixels up and to the right. That area may be in the blind spot. *Gated PixelCNN* fixes this with two streams. A vertical stack sees every row above at full width. A horizontal stack sees the current row to the left.

Concepts used

PixelCNN; masked convolution; causal constraint; Mask A vs. Mask B; receptive field; blind spot; Gated PixelCNN (vertical and horizontal stacks) (Lecture 2, slides "Masked convolution", "Mask A and Mask B", "The blind spot" and "Gated PixelCNN").

Problem 10 (Week 2): Colour channels inside one pixel

Question

For one pixel with channels (r, g, b) and past pixels $x_{<i}$:

1. Write the PixelRNN product $p(r, g, b \mid x_{<i})$.
2. Why is $p(r \mid x_{<i})\, p(g \mid x_{<i})\, p(b \mid x_{<i})$ a worse model? What loss, summed over the three channels, equals $-\log p_\theta$ at that pixel?

What the question is asking

A colour pixel has three numbers: red, green and blue, each from 0 to 255. When the model generates pixel i, it already knows all earlier pixels $x_{<i}$. The question asks how to write the probability of all three colour values together. Then it asks why the simpler version, where each colour is predicted separately, is worse, and which training loss matches the correct version.

Solution and explanation

Part 1: the PixelRNN product

The idea. Apply the chain rule *inside* the pixel too, with the fixed order red, then green, then blue:

$$p(r, g, b \mid x_{<i}) = p(r \mid x_{<i})\; p(g \mid r, x_{<i})\; p(b \mid r, g, x_{<i}).$$

Red looks only at the earlier pixels. Green also looks at the red value just chosen. Blue looks at both red and green.

Part 2: why the independent version is worse, and the matching loss

Why worse. The version $p(r \mid x_{<i})\, p(g \mid x_{<i})\, p(b \mid x_{<i})$ assumes that once you know the earlier pixels, the three colours of this pixel have nothing to do with each other. That is false for real images. The colour channels of one pixel are strongly linked.

Generative example. The model is generating a face and reaches a cheek pixel. A skin-coloured pixel has red, green and blue values that rise and fall together: bright skin has all three high, shaded

skin has all three lower. Suppose each channel is drawn on its own. Red might come out "bright" while green comes out "shaded" and blue "bright", giving a purple or greenish speck that no real face has. The chain-rule version avoids this: once red is chosen as "bright", green is predicted *knowing* that, so it picks a matching value.

Put differently, the chain rule is exact, so the conditional version can represent any colour relationship. The independent version is an extra assumption that throws that relationship away. It can only score real images as well or worse.

The matching loss. Each factor is a 256-way softmax, a probability for each of the 256 intensity values. The loss for one channel is the *cross-entropy*: minus the log of the probability the model gave to the true value. Sum it over the three channels, each conditioned on the channels before it:

$$-\log p_\theta(r \mid x_{<i}) \;-\; \log p_\theta(g \mid r, x_{<i}) \;-\; \log p_\theta(b \mid r, g, x_{<i}) \;=\; -\log p_\theta(r, g, b \mid x_{<i}).$$

The equality holds because the log turns the product from Part 1 into a sum. Add this over every pixel of the image and you get $-\log p_\theta(x)$ for the whole image, which is exactly what training minimizes.

Concepts used

Chain rule within a pixel (channel ordering R $\rightarrow$ G $\rightarrow$ B); PixelRNN output head; 256-way softmax; cross-entropy as negative log-likelihood; dependence between channels (Lecture 2, slides "Channel order inside one pixel" and "PixelRNN output head").

Week 3 setup: the pen story

Problems 11–15 all use the same tiny model from Lecture 3:

$$p(z) = \mathcal{N}(0, 1), \qquad p(x \mid z) = \mathcal{N}(z, 1), \qquad \text{observed } x = 4.$$

In words. A hidden number z is drawn from a bell curve centred at 0 with variance 1. This is the *prior*. Then the observed number x is z plus some noise, again with variance 1. This is the *decoder*. In the lecture, z is the hidden pull on a pen and x is where the pen lands.

The same thing in generative-model terms. Think of z as the hidden code a VAE uses, for example a hidden "brightness setting". x is the brightness of the image the decoder actually produces, which is the setting plus a little random noise. We observed one output, $x = 4$.

This model is simple enough that every exact answer can be computed. That lets us check the VAE approximations against the truth. With a real neural decoder we could not do that.

Problem 11 (Week 3): Exact marginal and posterior

Question

1. Show that $p(x) = \mathcal{N}(0, 2)$, and compute $\log p(4)$ from

$$\log p(x) = -\tfrac{1}{2}\log(4\pi) - \frac{x^2}{4}.$$

2. State the true posterior $p(z \mid x = 4)$: mean and variance.

3. To draw a new landing, do you sample z from $p(z)$ or from $q_\phi(z \mid x = 4)$?

What the question is asking

Part 1: what is the overall distribution of outputs x once the hidden z is averaged out, and how likely is the value 4? Part 2: having seen $x = 4$, what should we now believe about the hidden z? Part 3: to generate a fresh output, where should the hidden code come from: the prior, or the encoder's guess for the old observation?

Solution and explanation

Part 1: the marginal $p(x)$ and $\log p(4)$

The idea. Write the model as a sum: $x = z + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, 1)$ is noise unrelated to z. Adding two independent bell curves gives another bell curve. Its centre is the sum of the centres and its variance is the sum of the variances.

Mean:

$$\mathbb{E}[x] = \mathbb{E}[z] + \mathbb{E}[\varepsilon] = 0 + 0 = 0.$$

Variance: variances add because z and ε are independent:

$$\mathrm{Var}(x) = \mathrm{Var}(z) + \mathrm{Var}(\varepsilon) = 1 + 1 = 2.$$

So $p(x) = \mathcal{N}(0, 2)$. Intuitively, outputs spread out more than the hidden codes do, because two sources of randomness are stacked.

Where the given log formula comes from. A Gaussian with mean 0 and variance 2 has density $\frac{1}{\sqrt{2\pi \cdot 2}} e^{-x^2/(2\cdot 2)}$. Taking the log gives $-\frac{1}{2}\log(4\pi) - \frac{x^2}{4}$.

Plug in $x = 4$. First split $\frac{1}{2}\log(4\pi)$ into known constants:

$$\tfrac{1}{2}\log(4\pi) = \tfrac{1}{2}\log(2\pi) + \tfrac{1}{2}\log 2 \approx 0.919 + 0.347 = 1.266.$$

Then

$$\log p(4) = -1.266 - \frac{16}{4} = -1.266 - 4 = -5.266.$$

This is the exact score. Problem 15 compares the VAE's estimates against it.

Part 2: the true posterior $p(z \mid x = 4)$

The idea. By Bayes' rule, the posterior is proportional to prior × likelihood. Both are bell curves, so we multiply them and read off the new centre and spread.

Step 1. Keep only the parts that depend on z. Constants do not change the shape.

$$p(z \mid x) \propto \exp\Big(-\frac{(x-z)^2}{2}\Big)\exp\Big(-\frac{z^2}{2}\Big) = \exp\Big(-\frac{(x-z)^2}{2} - \frac{z^2}{2}\Big).$$

Step 2. Expand and complete the square:

$$-\frac{(x-z)^2}{2} - \frac{z^2}{2} = -z^2 + xz - \frac{x^2}{2} = -\Big(z - \frac{x}{2}\Big)^2 - \frac{x^2}{4}.$$

The last term has no z, so it is just a constant.

Step 3. Compare with the bell-curve shape $\exp\big(-\frac{(z-\mu)^2}{2\sigma^2}\big)$. The centre is $\mu = x/2$, and $2\sigma^2 = 1$, so $\sigma^2 = 1/2$.

At $x = 4$:

$$p(z \mid x = 4) = \mathcal{N}\big(2, \tfrac{1}{2}\big): \quad \text{mean } 2, \text{ variance } \tfrac{1}{2}.$$

Reading the answer. The prior pulls z toward 0. The observation pulls z toward 4. Both have equal strength (variance 1 each), so the best guess lands halfway, at 2. Our uncertainty shrinks from 1 to $\frac{1}{2}$ because we now have two sources of information instead of one.

Part 3: where to sample z for a new output

Answer: from the prior $p(z) = \mathcal{N}(0, 1)$. Then draw $x \sim p(x \mid z)$.

Why. A new output is a fresh run of the generative story, and the story starts with the prior. The encoder $q_\phi(z \mid x = 4)$ answers a different question: "what code probably produced *this particular* output 4?". If you sample from it, you get outputs near 4 again, a variation of the old one rather than a new one. For example, an image VAE that always sampled codes from the encoder of one training face would keep producing near-copies of that face. The encoder is used only when you already have an x in hand and want to encode it.

Concepts used

Sum of independent Gaussians; marginal likelihood; Bayes' rule; completing the square; true posterior; prior vs. encoder (approximate posterior); generation by ancestral sampling from the prior (Lecture 3, slides "This story still has a closed form" and "What a training step does"; Lecture 3 hand notes, Section 2).

Problem 12 (Week 3): Deriving the ELBO and its gap

Question

For any $q(z \mid x) > 0$:

1. Insert q into $\log p(x) = \log \int p(x, z)\, dz$ and apply Jensen's inequality to obtain a lower bound.
2. Split the bound into a reconstruction term and $\mathrm{KL}(q(z \mid x) \,\|\, p(z))$.
3. Write the identity that isolates the gap. Which KL is the gap, and when is the bound tight?

What the question is asking

With a real neural decoder we cannot compute $\log p(x)$ (Problem 4). A VAE instead computes a number that is guaranteed to be *at most* $\log p(x)$, called the ELBO (Evidence Lower BOund), and maximizes that. The question asks us to derive it, split it into two meaningful parts, and find exactly how far below $\log p(x)$ it sits.

$q(z \mid x)$ is the *encoder*: a simple distribution (a Gaussian) that guesses which codes produced x.

Solution and explanation

Part 1: Jensen's inequality gives a lower bound

Step 1: multiply and divide by q. This changes nothing, since $q/q = 1$:

$$\log p(x) = \log \int q(z \mid x)\, \frac{p(x, z)}{q(z \mid x)}\, dz.$$

Step 2: recognize an average. $\int q(z \mid x)\, [\text{something}]\, dz$ is the average of "something" when z is drawn from q. Write it as an expectation:

$$\log p(x) = \log \mathbb{E}_q \left[\frac{p(x, z)}{q(z \mid x)} \right].$$

Step 3: apply Jensen's inequality. The logarithm is a curve that bends downward (concave). For such a curve, the log of an average is always at least the average of the logs. A quick check with two numbers, 2 and 8:

$$\log \left(\tfrac{2+8}{2} \right) = \log 5 \approx 1.609 \;\geq\; \tfrac{1}{2} \log 2 + \tfrac{1}{2} \log 8 \approx 1.386.$$

So we may move the log inside, at the cost of turning "=" into "≥":

$$\log p(x) \;\geq\; \mathbb{E}_q \left[\log \frac{p(x, z)}{q(z \mid x)} \right] = \text{ELBO}.$$

Why this helps. The left side has a log *outside* an integral we cannot do. The right side is an average of simple terms. We can estimate it by drawing a few z from q and averaging.

Part 2: reconstruction minus KL

Step 1. Split the joint with the generative story, $p(x, z) = p(x \mid z)\, p(z)$:

$$\text{ELBO} = \mathbb{E}_q [\log p(x \mid z) + \log p(z) - \log q(z \mid x)]\,.$$

Step 2. Group the last two terms. $\mathbb{E}_q[\log q - \log p(z)]$ is by definition $\mathrm{KL}(q(z \mid x) \| p(z))$, so

$$\text{ELBO} = \underbrace{\mathbb{E}_{q(z|x)}[\log p(x \mid z)]}_{\text{reconstruction}} \;-\; \underbrace{\mathrm{KL}\big(q(z \mid x) \,\|\, p(z)\big)}_{\text{stay close to the prior}}.$$

What each term does, for an image VAE.

- *Reconstruction*: encode a face into a code, decode it back, and ask how likely the original face is under the decoder's output. It is high when the decoder can rebuild the input from the code.
- *KL to the prior*: a penalty for encoder outputs that drift far from $\mathcal{N}(0,1)$. It keeps all the codes in the region where we will later sample new codes. Without it, the decoder might only work on codes that $\mathcal{N}(0,1)$ never produces.

In code we minimize $-\text{ELBO}$ = reconstruction loss + KL loss.

Part 3: the gap

Step 1. Bayes' rule gives $p(x) = p(x,z)/p(z \mid x)$ for every z. So $\log p(x)$ equals its own average under q (it has no z in it):

$$\log p(x) = \mathbb{E}_q\left[\log \frac{p(x,z)}{p(z \mid x)}\right].$$

Step 2. Multiply and divide by q inside the log, and split:

$$\log p(x) = \mathbb{E}_q\left[\log \frac{p(x,z)}{q(z \mid x)}\right] + \mathbb{E}_q\left[\log \frac{q(z \mid x)}{p(z \mid x)}\right].$$

Step 3. Name the pieces:

$$\log p(x) = \text{ELBO} + \text{KL}\big(q(z \mid x) \,\|\, p(z \mid x)\big).$$

Which KL is the gap? The KL between the encoder and the *true posterior* $p(z \mid x)$. It is *not* the KL to the prior $p(z)$; that one stays inside the ELBO.

When is the bound tight? A KL is never negative, and it is zero only when the two distributions are identical. So ELBO $= \log p(x)$ exactly when $q(z \mid x) = p(z \mid x)$, meaning the encoder guesses the hidden code perfectly. (In Jensen terms: equality needs the ratio $p(x,z)/q(z \mid x)$ to be the same for every z, which happens exactly in this case.)

Plain-English summary. Maximizing the ELBO does two things at once: it makes the model better (raises $\log p(x)$), and it makes the encoder's guess better (shrinks the gap).

Concepts used

Approximate posterior (encoder); expectation; Jensen's inequality and concavity of log; ELBO; reconstruction term; KL to the prior; ELBO–evidence identity; tightness when q equals the true posterior (Lecture 3, slides "Jensen, on two numbers", "From the integral to the ELBO" and "The gap is a KL").

Problem 13 (Week 3): The Gaussian KL and its derivatives

Question

For $q = \mathcal{N}(\mu, \sigma^2)$ and $p = \mathcal{N}(0,1)$,

$$\text{KL}(q\|p) = \tfrac{1}{2}\left(\mu^2 + \sigma^2 - \log\sigma^2 - 1\right).$$

1. Evaluate the KL at $(\mu, \sigma^2) = (0,1)$ and at $(\mu, \sigma^2) = (2,1)$.
2. Derive $\partial\text{KL}/\partial\mu$ and $\partial\text{KL}/\partial\sigma$.
3. If the network emits $\ell = \log\sigma^2$, show $\partial\text{KL}/\partial\ell = \frac{1}{2}(\sigma^2 - 1)$.

What the question is asking

For each input, the VAE encoder outputs a centre μ and a spread σ^2 for its guessed code distribution. The KL formula measures how far that guess is from the prior $\mathcal{N}(0,1)$. The question asks us to evaluate it at two settings, and to find its slopes (derivatives). Those slopes are what backpropagation uses to adjust the encoder.

Solution and explanation

Part 1: two evaluations

At $(\mu, \sigma^2) = (0, 1)$: the encoder output is exactly the prior.

$$\mathrm{KL} = \tfrac{1}{2}(0 + 1 - \log 1 - 1) = \tfrac{1}{2}(0 + 1 - 0 - 1) = 0.$$

Identical distributions have zero distance.

At $(\mu, \sigma^2) = (2, 1)$: same spread, but the centre is moved to 2.

$$\mathrm{KL} = \tfrac{1}{2}(4 + 1 - 0 - 1) = \tfrac{1}{2}(4) = 2.$$

When the spread matches, the KL is just $\frac{1}{2}\mu^2$: it grows with the square of how far the code's centre is pushed from 0.

Part 2: slopes with respect to μ and σ

With respect to μ. Only the μ^2 term contains μ:

$$\frac{\partial\,\mathrm{KL}}{\partial \mu} = \tfrac{1}{2} \cdot 2\mu = \mu.$$

Meaning: when we minimize the loss, this gradient always pulls the code's centre back toward 0, harder the further away it is.

With respect to σ. Write the σ terms with σ itself: σ^2 and $-\log \sigma^2 = -2\log\sigma$. Their slopes are 2σ and $-2/\sigma$:

$$\frac{\partial\,\mathrm{KL}}{\partial \sigma} = \tfrac{1}{2}\Big(2\sigma - \frac{2}{\sigma}\Big) = \sigma - \frac{1}{\sigma}.$$

Meaning:

- at $\sigma = 1$ the slope is $1 - 1 = 0$, so the KL is happiest at the prior's spread;
- if $\sigma < 1$ (the encoder is too sure of itself) the slope is negative, so minimizing pushes σ up;
- if $\sigma > 1$ the slope is positive, so minimizing pushes σ down.

Part 3: slope with respect to the log-variance ℓ

Why networks output $\ell = \log \sigma^2$. A network's raw output can be any number, including negative ones, but a variance must be positive. Setting $\sigma^2 = e^{\ell}$ guarantees $\sigma^2 > 0$ for any ℓ.

Step 1. Rewrite the KL in terms of ℓ, using $\sigma^2 = e^{\ell}$ and $\log \sigma^2 = \ell$:

$$\mathrm{KL} = \tfrac{1}{2}\left(\mu^2 + e^{\ell} - \ell - 1\right).$$

Step 2. Differentiate. The slope of e^{ℓ} is e^{ℓ}, and the slope of ℓ is 1:

$$\frac{\partial\,\mathrm{KL}}{\partial \ell} = \tfrac{1}{2}\left(e^{\ell} - 1\right) = \tfrac{1}{2}\left(\sigma^2 - 1\right). \qquad \blacksquare$$

It is zero at $\sigma^2 = 1$, the same resting point as in Part 2. Changing the way we parametrize the spread does not change where the KL is smallest.

Concepts used

Closed-form KL between Gaussians; KL as distance to the prior; partial derivatives; log-variance parameterization (Lecture 3, slides "Derive the one-dimensional KL" and "Differentiate the KL").

Problem 14 (Week 3): Reparameterization and reconstruction gradients

Question

The decoder is $p(x \mid z) = \mathcal{N}(z, 1)$, and $z = \mu + \sigma\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$.

1. Write $\partial z/\partial\mu$ and $\partial z/\partial\sigma$.
2. Show $\partial \log p(x \mid z)/\partial z = x - z$.
3. At $x = 4$, $\mu = 1$, $\sigma = 2$, $\varepsilon = 1$, compute z and the two reconstruction slopes $\partial \log p/\partial\mu$ and $\partial \log p/\partial\sigma$.
4. If instead $\varepsilon = 0$ on every step, which parameter receives no reconstruction gradient?

What the question is asking

To train the encoder, we need to know how the reconstruction score changes when we nudge the encoder's outputs μ and σ. But z is a *random* draw, and you cannot take the slope of a dice-like random choice directly. The *reparameterization trick* solves this. Draw a plain noise value ε first, then build $z = \mu + \sigma\varepsilon$, which is an ordinary formula with a slope.

The question asks for the slopes along the chain $\mu, \sigma \to z \to \log p(x \mid z)$, a numerical example, and what happens if the noise is switched off.

Solution and explanation

Part 1: how z responds to μ and σ

$z = \mu + \sigma\varepsilon$ is a straight line in each variable, with ε treated as a fixed number once drawn:

$$\frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \varepsilon.$$

Moving the centre by 1 moves the code by 1. Widening the spread moves the code by however much noise was drawn. Note that $z = \mu + \sigma\varepsilon$ still has the right distribution, $\mathcal{N}(\mu, \sigma^2)$. We only rewrote *how* it is drawn.

Part 2: slope of the reconstruction with respect to z

The log of the decoder's bell curve is

$$\log p(x \mid z) = -\tfrac{1}{2}\log(2\pi) - \tfrac{1}{2}(x - z)^2.$$

The first term is a constant, so its slope is 0. For the second term, by the chain rule, the slope of $(x - z)^2$ with respect to z is $2(x - z) \cdot (-1)$. So

$$\frac{\partial}{\partial z} \log p(x \mid z) = -\tfrac{1}{2} \cdot 2(x - z) \cdot (-1) = x - z. \qquad \blacksquare$$

Meaning: if the real output x is to the right of the code ($x > z$), the slope is positive, so moving z right improves the reconstruction. It says "move the code toward the data".

Part 3: the numbers

The code:

$$z = \mu + \sigma\varepsilon = 1 + 2 \times 1 = 3.$$

Slope at z:

$$x - z = 4 - 3 = 1.$$

Chain rule to μ and σ: multiply the slope at z by how z responds:

$$\frac{\partial \log p}{\partial \mu} = \frac{\partial \log p}{\partial z} \cdot \frac{\partial z}{\partial \mu} = 1 \times 1 = 1, \qquad \frac{\partial \log p}{\partial \sigma} = \frac{\partial \log p}{\partial z} \cdot \frac{\partial z}{\partial \sigma} = 1 \times \varepsilon = 1.$$

Both slopes are $+1$. On this one sample, raising μ or σ would improve the reconstruction.

Two extra points (from Lecture 3).

- *One noise draw is a noisy hint.* The *average* reconstruction slope for σ over all possible ε is $-\sigma = -2$: on average a wider spread hurts reconstruction. This single draw happened to say $+1$. VAEs normally use one ε per data point and rely on many training steps to average out this noise.
- *The full loss also has the KL.* The loss is $-\log p(x \mid z) + \mathrm{KL}$. At $\mu = 1$, $\sigma = 2$, Problem 13's formulas give $\partial\mathrm{KL}/\partial\mu = 1$ and $\partial\mathrm{KL}/\partial\sigma = 2 - \frac{1}{2} = 1.5$. So the total loss slopes on this sample are $-1 + 1 = 0$ for μ and $-1 + 1.5 = 0.5$ for σ.

Part 4: what if $\varepsilon = 0$ always?

Answer: σ receives no reconstruction gradient.

Why. $\partial z/\partial\sigma = \varepsilon = 0$, so the chain-rule product $(x - z) \cdot 0$ is always zero. With no noise, $z = \mu$ every time and σ has no effect on the code, so the reconstruction cannot tell σ anything. μ still learns, since $\partial z/\partial\mu = 1$. The only signal left for σ is the KL term, which just pushes it toward 1. At that point we are no longer training the ELBO. The model has turned into a plain (non-random) autoencoder with a decorative σ, and it never learns how uncertain its codes should be.

Concepts used

Reparameterization trick; chain rule of calculus (backpropagation through z); gradient of a Gaussian log-likelihood; single-sample (Monte Carlo) gradient estimate and its noise; combined loss $-$ELBO (Lecture 3, slides "Differentiate the reconstruction", "A sample is not a derivative" and "Chain rule, one sample").

Problem 15 (Week 3): Two encoders, two ELBOs

Question

Same model as Problem 11. Use

$$\mathbb{E}_q[\log p(x \mid z)] = -0.919 - \tfrac{1}{2}\mathbb{E}_q[(x - z)^2], \qquad \mathbb{E}_q[(x - z)^2] = \sigma^2 + (x - \mu)^2,$$

and $\log p(4) \approx -5.266$ from Problem 11. At $x = 4$:

1. For the loose encoder $q = \mathcal{N}(0, 1)$, compute the reconstruction, $\mathrm{KL}(q \| p(z))$, the ELBO, and the gap $\log p(4) - \mathrm{ELBO}$.
2. For the matched encoder $q = \mathcal{N}(2, 1/2)$, compute the same four quantities. For the KL, use $\log(1/2) = -\log 2 \approx -0.693$.
3. Which KL equals the gap: the KL against $p(z)$, or the KL against $p(z \mid x)$?

What the question is asking

We try two different encoders on the same observation $x = 4$, compute the ELBO for each, and compare it with the true $\log p(4) = -5.266$.

- The *loose* encoder ignores the data completely and always answers $\mathcal{N}(0, 1)$, the prior.
- The *matched* encoder answers $\mathcal{N}(2, \frac{1}{2})$, which is exactly the true posterior from Problem 11.

The goal is to see the ELBO's two parts and its gap as actual numbers.

Solution and explanation

The given helper formula. $\mathbb{E}_q[(x - z)^2] = \sigma^2 + (x - \mu)^2$ says: the average squared distance between the data and a random code is the code's spread, plus the squared distance between the data and the code's centre. Both being uncertain and being off-centre cost you.

Part 1: the loose encoder $q = \mathcal{N}(0, 1)$

Here $\mu = 0$ and $\sigma^2 = 1$.

Reconstruction.

$$\mathbb{E}_q[(4 - z)^2] = 1 + (4 - 0)^2 = 17,$$

$$\text{Recon} = -0.919 - \tfrac{1}{2}(17) = -0.919 - 8.5 = -9.419.$$

This is very poor. The codes cluster around 0, but the data is at 4, so the decoder's outputs are far from the data.

KL to the prior. q is the prior, so

$$\text{KL}(q \| p(z)) = \tfrac{1}{2}(0 + 1 - 0 - 1) = 0.$$

ELBO.

$$\text{ELBO} = -9.419 - 0 = -9.419.$$

Gap.

$$\log p(4) - \text{ELBO} = -5.266 - (-9.419) = 4.153.$$

Check against the true posterior. Using the KL formula for two Gaussians from the Lecture 3 hand notes, with $q = \mathcal{N}(0, 1)$ and $p(z \mid x) = \mathcal{N}(2, \frac{1}{2})$:

$$\text{KL}\big(q \,\|\, p(z \mid x)\big) = \tfrac{1}{2}\left[\log \frac{0.5}{1} + \frac{1 + (0 - 2)^2}{0.5} - 1\right] = \tfrac{1}{2}\left[-0.693 + 10 - 1\right] \approx 4.153.$$

It matches the gap.

Part 2: the matched encoder $q = \mathcal{N}(2, \frac{1}{2})$

Here $\mu = 2$ and $\sigma^2 = \frac{1}{2}$.

Reconstruction.

$$\mathbb{E}_q[(4 - z)^2] = 0.5 + (4 - 2)^2 = 4.5,$$

$$\text{Recon} = -0.919 - \tfrac{1}{2}(4.5) = -0.919 - 2.25 = -3.169.$$

Much better. The codes are now centred where they actually explain the data.

KL to the prior.

$$\text{KL} = \tfrac{1}{2}\left(2^2 + 0.5 - \log 0.5 - 1\right) = \tfrac{1}{2}\left(4 + 0.5 + 0.693 - 1\right) = \tfrac{1}{2}(4.193) \approx 2.097.$$

Now we pay a penalty, because the encoder moved away from the prior.

ELBO.

$$\text{ELBO} = -3.169 - 2.097 = -5.266.$$

Gap.

$$\log p(4) - \text{ELBO} = -5.266 - (-5.266) = 0.$$

The bound is exact.

Side by side.

	Loose $q = \mathcal{N}(0, 1)$	Matched $q = \mathcal{N}(2, \frac{1}{2})$
Reconstruction	-9.419	-3.169
$\text{KL}(q\|\|p(z))$	0	2.097
ELBO	-9.419	-5.266
$\log p(4)$	-5.266	-5.266
Gap	4.153	0

Reading the table. The loose encoder pays nothing to the prior, yet it loses badly. Its reconstruction is terrible, and its ELBO underestimates the truth by more than 4. The matched encoder pays about 2.1 to the prior, but gains over 6 in reconstruction, and its ELBO hits the true value exactly. Note that $\log p(4)$ is the same in both columns: it belongs to the model, not to the encoder. Only the ELBO changes.

A lesson for real VAEs. If someone reports the ELBO of a poor encoder as if it were $\log p(x)$, they are reporting -9.419 when the truth is -5.266. For image VAEs we usually cannot compute the gap, so the ELBO should always be described as a lower bound.

Part 3: which KL is the gap?

Answer: the KL against the true posterior, $\text{KL}\big(q(z \mid x) \,\|\, p(z \mid x)\big)$**.**

The numbers prove it:

- Loose encoder: KL to the prior = 0, but gap = 4.153. So the gap is not the KL to the prior. It equals the KL to the posterior, 4.153 (checked above).
- Matched encoder: KL to the prior = 2.097, but gap = 0. The gap is zero because q *is* the posterior, so $\text{KL}(q\|\|p(z \mid x)) = 0$.

The KL to the prior is a cost *inside* the ELBO. The KL to the posterior is the distance *between* the ELBO and $\log p(x)$.

Concepts used

ELBO = reconstruction − KL to prior; expected squared error under a Gaussian; closed-form Gaussian KL; ELBO–evidence identity, gap = $\text{KL}(q\|\|p(z \mid x))$; tightness when the encoder matches the true posterior (Lecture 3, slide "Two encoders for the landing"; Lecture 3 hand notes, Section 11).